

Guide des Makefiles JobbingTrack

Documentation complète du système de Makefiles unifié de JobbingTrack.

Vue d'Ensemble

Le système de Makefiles de JobbingTrack est organisé de manière hiérarchique pour une gestion optimale du développement et du déploiement.

```
JobbingTrack/  
├─  Makefile           # Makefile principal (point d'entrée)  
├─  makefiles/  
│   └─  backend/           # Makefiles spécifiques au backend  
│   └─  frontend/         # Makefiles spécifiques au frontend  
│   └─  shared/           # Fonctions et variables communes  
│   └─  tests/            # Makefiles pour les tests  
└─  scripts/          # Scripts shell complémentaires
```

Makefile Principal

Commandes de Base

Démarrage et Arrêt

```
make up           # Démarrer tout (backend + frontend + données)  
make down         # Arrêter tout proprement  
make dev          # Mode développement avec hot reload  
make build        # Construire toutes les images
```

Maintenance

```
make clean        # Nettoyer complètement (containers + volumes + images)  
make rebuild      # Reconstruction complète  
make fix          # Diagnostic + correction automatique
```

```
make status      # État de tous les services
make logs        # Logs temps réel
```

Tests

```
make test-all    # Tous les tests (unitaires + E2E + intégration)
make test-e2e     # Tests end-to-end Playwright
make test-services # Tests de santé des microservices
```

Commandes Avancées

Diagnostic et Prévention

```
make diagnose     # Diagnostic complet préventif
make check-deps   # Vérifier toutes les dépendances
make check-health  # Vérification santé préventive
make backup       # Sauvegarde complète du projet
```

Développement

```
make migrate      # Appliquer les migrations BDD
make create-admin  # Créer l'utilisateur administrateur
make install      # Installation complète
```

Sous-Makefiles

Backend (`makefiles/backend/`)

Commandes Spécialisées Backend

```
make build-backend # Construire uniquement le backend
make up-backend    # Démarrer uniquement le backend
make down-backend  # Arrêter uniquement le backend
make logs-backend  # Logs backend uniquement
make test-backend  # Tests backend uniquement
```

Services Individuels

```
make start-auth-service      # Démarrer le service d'authentification
make stop-application-service # Arrêter le service des candidatures
make restart-company-service  # Redémarrer le service des entreprises
make rebuild-contact-service  # Reconstruire le service des contacts
```

Frontend (`makefiles/frontend/`)

Commandes Spécialisées Frontend

```
make build-frontend      # Construire uniquement le frontend
make up-frontend         # Démarrer uniquement le frontend
make down-frontend       # Arrêter uniquement le frontend
make logs-frontend       # Logs frontend uniquement
make test-frontend       # Tests frontend uniquement
```

Développement Frontend

```
make dev-frontend      # Mode développement frontend
make build-frontend     # Construire le frontend
make test-e2e-frontend  # Tests E2E frontend
```

Tests (`makefiles/tests/`)

Tests Spécialisés

```
make test-unit          # Tests unitaires uniquement
make test-integration    # Tests d'intégration uniquement
make test-load           # Tests de charge uniquement
make test-security       # Tests de sécurité uniquement
```

Fonctions Communes (`makefiles/shared/`)

Fonctions Utilitaires

Vérification et Validation

```
# Vérifier les dépendances système
check_dependencies = @echo "🔍 Vérification des dépendances..." && \
  if ! command -v docker &> /dev/null; then \
    echo "❌ Docker n'est pas installé"; exit 1; \
  fi && \
  if ! command -v docker-compose &> /dev/null; then \
    echo "❌ Docker Compose n'est pas installé"; exit 1; \
  fi && \
  echo "✅ Toutes les dépendances sont installées"

# Attendre que PostgreSQL soit prêt
wait_for_postgres = @echo "⌚ Attente de PostgreSQL..." && \
  for i in $(seq 1 30); do \
    if docker-compose exec postgres pg_isready -U jobbingtrack > /dev/null 2>&: \
      echo "✅ PostgreSQL est prêt"; break; \
    fi; \
    sleep 2; \
    if [ $$i -eq 30 ]; then \
      echo "❌ Timeout: PostgreSQL non accessible"; exit 1; \
    fi; \
  done

# Messages colorés
print_message = @echo "$(1) 🚀 $(2)$(3)"
print_section = @echo "$(1)===== $(2)"
```

Variables Communes

```
# Configuration
BACKEND_DIR = ../backend
FRONTEND_DIR = ../frontend
SCRIPTS_DIR = ../scripts
TESTS_DIR = ../tests

# Services
SERVICES = api-gateway auth-service application-service company-service contact-service
BACKOFFICE_SERVICES = postgres redis auth-service dashboard-service api-gateway
FULL_SERVICES = postgres redis auth-service application-service company-service contact-service

# Couleurs
GREEN = \033[0;32m
RED = \033[0;31m
YELLOW = \033[1;33m
BLUE = \033[0;34m
PURPLE = \033[0;35m
```

```
CYAN = \033[0;36m
```

```
NC = \033[0m
```

Scripts Shell (`scripts/`)

Structure des Scripts

```
scripts/
├─ 📁 database/                # Scripts de gestion BDD
│   ├── create-admin-user.sh  # Création utilisateur admin
│   ├── backup.sh             # Sauvegarde automatique
│   └─ restore-backup.sh      # Restauration depuis backup
├─ 📁 deployment/              # Scripts de déploiement
│   ├── diagnostic-fix.sh     # Diagnostic + correction
│   ├── start-all.sh         # Démarrage complet
│   └─ stop-all.sh           # Arrêt complet
├─ 📁 monitoring/              # Scripts de monitoring
│   ├── health-check.sh       # Vérification santé
│   ├── metrics.sh            # Collecte métriques
│   └─ alerts.sh              # Configuration alertes
├─ 📁 system/                  # Scripts système
│   ├── setup.sh              # Installation initiale
│   ├── cleanup.sh            # Nettoyage système
│   └─ verify.sh              # Vérification intégrité
└─ 📁 testing/                 # Scripts de tests
    ├── run-all.sh           # Exécution tous tests
    └─ performance.sh         # Tests performance
```

Utilisation des Scripts

Diagnostic et Correction

```
# Diagnostic complet
./scripts/deployment/diagnostic-fix.sh full

# Correction automatique
./scripts/deployment/diagnostic-fix.sh fix

# Diagnostic uniquement
./scripts/deployment/diagnostic-fix.sh check
```

Base de Données

```
# Créer l'utilisateur admin
./scripts/database/create-admin-user.sh

# Sauvegarde
./scripts/database/backup.sh

# Restauration
./scripts/database/restore-backup.sh backup_20250101.sql
```

Bonnes Pratiques

Développement

- Utiliser le **Makefile principal** pour toutes les opérations
- **Préfixer les commandes spécifiques** avec le nom du module
- Utiliser les **fonctions communes** pour la cohérence

Déploiement

- **Tester localement** avant déploiement
- Utiliser les **scripts de diagnostic** avant production
- **Sauvegarder** avant les opérations destructives

Maintenance

- **Surveiller les logs** régulièrement
- **Vérifier la santé** des services quotidiennement
- **Mettre à jour** les dépendances régulièrement



Documentation des Commandes

Commandes Backend (`make *-backend`)

- `build-backend` : Construire uniquement le backend
- `up-backend` : Démarrer uniquement le backend
- `down-backend` : Arrêter uniquement le backend
- `logs-backend` : Logs backend uniquement
- `test-backend` : Tests backend uniquement

Commandes Frontend (`make *-frontend`)

- `build-frontend` : Construire uniquement le frontend
- `up-frontend` : Démarrer uniquement le frontend
- `down-frontend` : Arrêter uniquement le frontend
- `logs-frontend` : Logs frontend uniquement
- `test-frontend` : Tests frontend uniquement

Commandes Tests (`make *-tests`)

- `test-unit-tests` : Tests unitaires uniquement
- `test-integration-tests` : Tests d'intégration uniquement
- `test-e2e-tests` : Tests end-to-end uniquement
- `test-performance-tests` : Tests de performance uniquement

Personnalisation

Ajout de Nouvelles Commandes

Dans le Makefile Principal

```
# Nouvelle commande personnalisée
my-command:
    @echo "Ma commande personnalisée"
    @cd $(BACKEND_DIR) && docker-compose exec api-gateway my-command
```

Dans un Sous-Makefile

```
# Dans makefiles/backend/custom.mk
.PHONY: custom-backend-command

custom-backend-command:
    @echo "Commande backend personnalisée"
    @docker-compose exec api-gateway custom-command
```

Variables Personnalisées

Dans `.env`

```
# Variables d'environnement personnalisées
MY_CUSTOM_VAR=value
```

```
MY_API_KEY=secret-key
```

Dans les Makefiles

```
# Utilisation des variables
custom-command:
    @echo "Utilisation de $(MY_CUSTOM_VAR)"
    @curl -H "Authorization: $(MY_API_KEY)" http://api.example.com
```

Résolution de Problèmes

Problèmes Courants

1. Services ne Démarrent Pas

```
# Diagnostic
make diagnose

# Correction
make fix
```

2. Erreurs de Build

```
# Nettoyer et reconstruire
make clean
make build

# Ou reconstruction complète
make rebuild
```

3. Problèmes de Base de Données

```
# Vérifier PostgreSQL
make check-health
```



```
# Recréer les données  
make create-admin
```

Debugging Avancé

Logs Détaillés

```
# Tous les logs avec timestamps  
make logs | ts  
  
# Logs d'un service spécifique  
make logs-auth-service  
  
# Logs avec filtrage  
make logs | grep -i error
```

Debugging Docker

```
# Inspecter un container  
docker inspect jobbingtrack-api-gateway  
  
# Voir les processus  
docker top jobbingtrack-api-gateway  
  
# Accéder au shell d'un container  
docker exec -it jobbingtrack-api-gateway /bin/sh
```



Métriques et Monitoring

Métriques Intégrées

Commandes de Monitoring

```
# Métriques de performance  
make check-health  
  
# Utilisation des ressources  
docker stats
```

```
# État du système  
make status
```

Scripts de Monitoring

```
# Surveillance continue  
./scripts/monitoring/health-monitor.sh  
  
# Collecte de métriques  
./scripts/monitoring/collect-metrics.sh  
  
# Alertes automatiques  
./scripts/monitoring/setup-alerts.sh
```

Apprentissage

Structure Hiérarchique

1. **Makefile Principal** : Interface utilisateur simple
2. **Sous-Makefiles** : Fonctionnalités spécialisées
3. **Scripts Shell** : Opérations complexes
4. **Variables Communes** : Cohérence globale

Évolution du Système

- **Modularité** : Ajout facile de nouvelles commandes
- **Maintenabilité** : Code organisé et documenté
- **Extensibilité** : Support de nouveaux environnements
- **Fiabilité** : Gestion d'erreurs robuste

 **Makefiles JobbingTrack** - Système de build et de déploiement unifié et extensible.

Navigation

-  [Index](#)
-  [Accueil](#)